Developer Handbook

How to use this handbook

Every developer reads and signs this handbook before writing code on any project. The rules are the same for everyone; what changes with level is how much you own. Use it alongside the Before Development and Before Launch checklists.

Area	Beginner (0–2 yrs)	Mid (2–5 yrs)	Senior (5+ yrs)
Code	Follows the rules in this handbook; small, clean tasks	Designs modules; writes code others can extend	Sets standards; designs systems; keeps code simple
Quality	Passes lint, tests and SonarQube before every push	Reviews others' PRs; improves coverage	Owns the quality gate; reduces tech debt every sprint
Testing	Writes unit tests for own code	Writes integration tests; tests edge cases	Defines test strategy; load and failure testing
Security	Never pushes secrets; validates input	Applies OWASP Top 10; reviews auth code	Threat models; owns secrets and access
Problem solving	Asks after 30 minutes stuck, with what was tried	Debugs across services independently	Finds root causes; prevents repeat incidents
AI tools	Uses Claude Code for learning and small tasks; reviews every line	Uses it for features with plans and tests	Sets AI rules, shared CLAUDE.md, reviews AI-heavy PRs
Communication	Daily update: done, doing, blocked	Writes clear PR descriptions and docs	Writes design docs; mentors juniors; talks to clients
Ownership	Owns tasks	Owns features	Owns outcomes in production

Daily workflow

Pull every morning, work on small branches, and push only code that passes lint, tests and SonarQube.

Every morning

- [ ] `git checkout main && git pull` before starting work
- [ ] Rebase or merge main into your feature branch: `git pull --rebase origin main`
- [ ] Fix conflicts locally and re-run tests before pushing
- [ ] Check the ticket, PR comments and failed builds from yesterday

Branches and commits

- [ ] Never commit directly to main, staging or production branches
- [ ] Branch name: `feature/JIRA-123-short-name`, `fix/JIRA-456-login-error`, `hotfix/...`
- [ ] Commit messages follow Conventional Commits: `feat:`, `fix:`, `refactor:`, `test:`, `docs:`, `chore:`
- [ ] Small commits that do one thing; no "final changes" or "update" messages
- [ ] Run `git status` and `git diff` before every commit; never `git add .` blindly

Before every push

```
npm run lint        # or: ruff check .
npm run test        # all tests green
npm run build       # it must build
# SonarQube runs in CI; check the result before asking for review
```

Pull requests

- [ ] PR under ~400 changed lines; split bigger work
- [ ] Description: what changed, why, how to test, screenshots for UI
- [ ] Linked ticket
- [ ] CI green, SonarQube quality gate passed, test report attached
- [ ] At least 1 approval (2 for auth, payment or infrastructure changes)
- [ ] Self-review your own diff before asking anyone else
- [ ] Reply to every review comment; never resolve someone else's comment without a fix or answer
- [ ] Delete the branch after merge

End of day

- [ ] Push work in progress to your branch (never leave work only on your laptop)
- [ ] Update the ticket status and write blockers in the team channel

Code quality rules

Code is read ten times more than it is written, so write it for the next person.

Size limits

Item	Limit	If you go over
File / page / component	400 lines max (aim for 200)	Split by responsibility into separate files
Function / method	40 lines max (aim for 20)	Extract smaller named functions
Function parameters	4 max	Pass one object instead
Nesting depth	3 levels max	Use early returns
Cognitive complexity (SonarQube)	15 per function	Simplify or split
Pull request	~400 changed lines	Split into several PRs

No hard-coding

- [ ] No URLs, IPs, ports, keys, passwords or email addresses in code; use config or environment variables
- [ ] No magic numbers or strings; use named constants (`MAX_UPLOAD_MB = 10`, not `10`)
- [ ] No hard-coded user IDs, tenant IDs or test data in production code
- [ ] Status values, roles and types live in one enum or constants file
- [ ] User-facing text in one place (i18n files), not scattered in components
- [ ] Feature switches through feature flags or config, not commented-out code

Clean code

- [ ] Clear names: `getUserOrders()`, not `getData()`; `isActive`, not `flag`
- [ ] One function does one thing; one file has one responsibility
- [ ] No duplicated code; repeat once is fine, three times means extract it (DRY)

- [ ] Early returns instead of deep if-else chains
- [ ] No commented-out code, unused imports, unused variables or console.log / print
- [ ] Comments explain *why*, not *what*
- [ ] Handle every error: no empty catch blocks; log details, show users a friendly message
- [ ] Validate all input on the server (Joi, Zod, Pydantic)
- [ ] Timeouts on every external call (HTTP, database, queue)
- [ ] Use async / await properly; no unhandled promises
- [ ] Strict typing: TypeScript `strict: true`, Python type hints
- [ ] Follow SOLID at the level that keeps code simple; do not over-engineer

Formatting

- [ ] Prettier + ESLint (JavaScript / TypeScript) or Black + Ruff (Python), auto-run on save
- [ ] Husky + lint-staged pre-commit hook so badly formatted code cannot be committed

Folder structure and architecture

Every project follows a layered structure: routes receive requests, controllers handle HTTP, services hold business logic, repositories talk to the database. Each layer only calls the layer below it.

Node.js backend

```
src/
  config/          # reads env vars, one place for all config
  modules/
    orders/
      orders.routes.ts       # URL -> controller
      orders.controller.ts   # HTTP in/out only, no business logic
      orders.service.ts      # business logic
      orders.repository.ts   # database queries only
      orders.validation.ts   # request schemas (Zod/Joi)
      orders.types.ts
      orders.service.test.ts
  middlewares/     # auth, error handler, rate limit, request ID
  queues/          # RabbitMQ producers and consumers
  utils/           # small shared helpers
  constants/       # enums, roles, status values
  app.ts
```

```
  server.ts
tests/              # integration and e2e tests
.env.example
Dockerfile
```

React frontend

```
src/
  app/              # routes, providers, layout
  features/
    orders/
      components/   # UI only
      hooks/        # data fetching and state
      api.ts        # calls to backend
      types.ts
  components/       # shared UI (Button, Modal)
  lib/              # api client, helpers
  config/           # env-based config
  constants/
  i18n/             # user-facing text
```

Python service

```
app/
  core/             # config, logging, security
  api/v1/           # routers
  services/         # business logic
  repositories/     # database access
  schemas/          # Pydantic models
  workers/          # queue consumers
tests/
pyproject.toml
.env.example
```

Architecture rules

- [ ] Group code by feature (orders, users, payments), not by file type across the whole app
- [ ] Controllers never contain SQL or business logic
- [ ] Services never read `req` / `res` or environment variables directly; config is passed in
- [ ] Database access only through repositories
- [ ] Shared code goes to `utils` or a shared package only when 2+ features need it
- [ ] One global error handler and one logger, used everywhere

- [ ] Long tasks go to queues, not HTTP requests
- [ ] Stateless services: no user data kept in server memory
- [ ] Every architectural decision written as a short ADR in `docs/adr/`

Config, .env, .gitignore and secrets

A secret pushed to Git is a leaked secret, even if you delete it a minute later; bots scan GitHub constantly.

Rules

- [ ] `.env` is never committed; only `.env.example` with fake placeholder values
- [ ] All config is read in one file (`src/config`) and validated at startup; the app refuses to start if a required value is missing
- [ ] Local: `.env` file. Staging / production: Azure Key Vault or Kubernetes secrets, injected as environment variables
- [ ] Different keys for local, staging and production; never reuse production keys anywhere else
- [ ] Never print secrets in logs, errors, screenshots, Slack, Teams, Jira or AI chats
- [ ] Never share secrets over WhatsApp or email; use a password manager or Key Vault access
- [ ] Secret scanning in pre-commit (Gitleaks) and in CI (GitHub secret scanning)
- [ ] If a secret leaks: tell the lead immediately, rotate the key first, then clean Git history

.gitignore (minimum)

```
# secrets
.env
.env.*
!.env.example
*.pem
*.key
*.p12
secrets/

# dependencies and builds
node_modules/
dist/
build/
__pycache__/
.venv/
```

```
coverage/

# local and editor files
.DS_Store
.idea/
.vscode/*
!.vscode/extensions.json
*.log

# local databases and data dumps
*.sqlite
*.dump
*.csv
```

.env.example

```
NODE_ENV=development
PORT=3000
DATABASE_URL=postgres://user:password@localhost:5432/app_dev
RABBITMQ_URL=amqp://guest:guest@localhost:5672
JWT_SECRET=change-me
AZURE_STORAGE_CONNECTION=change-me
```

Config loader pattern

```
// src/config/index.ts
import { z } from "zod";

const schema = z.object({
  NODE_ENV: z.enum(["development", "staging", "production"]),
  PORT: z.coerce.number().default(3000),
  DATABASE_URL: z.string().url(),
  JWT_SECRET: z.string().min(32),
});

export const config = schema.parse(process.env); // crashes early if config
is wrong
```

Keeping data safe across local, staging and production

Real customer data lives only in production. Everything else uses fake or masked data.

Environment	Data	Who has access	Rules
Local	Fake seed data only	Each developer	Never copy production data to a laptop
Dev / staging	Fake or masked copy	Dev team	Masked: names, phones, emails, addresses replaced; separate keys and databases
Production	Real customer data	Named leads only, read-only by default	Access logged, MFA, approval for any write

Rules for everyone

- [ ] Never download production databases, exports or logs to your laptop
- [ ] Never paste customer data into Claude, ChatGPT, Slack, Teams or Jira; use fake examples
- [ ] Production database access: read-only account by default; any write goes through a reviewed script
- [ ] Every production query has a WHERE clause and is reviewed; run it in a transaction first
- [ ] Seed scripts (`npm run seed`) create realistic fake data with Faker
- [ ] Staging is protected: login required, not indexed by Google, not public
- [ ] Staging never sends real emails, SMS or payments; use sandbox keys and test inboxes
- [ ] APIs return only the fields needed (DTOs); never return password hashes, tokens or internal IDs
- [ ] Logs never contain passwords, tokens, OTPs, card numbers, Aadhaar, phone numbers or full emails
- [ ] Debug mode, Swagger and test endpoints off in production
- [ ] Databases, Redis and RabbitMQ never exposed to the internet; private network only
- [ ] Lost laptop or suspected leak: inform the lead immediately (CERT-In reporting is 6 hours)
- [ ] Laptops use disk encryption, screen lock and updated OS

Unit testing and test reports

No PR is merged without tests for the new code and a passing test report.

Rules

- [ ] Every new service function, utility and bug fix comes with unit tests
- [ ] Every bug fix starts with a test that reproduces the bug
- [ ] Coverage on new code at least 80%; overall target 70%+ on core modules
- [ ] Test names describe behaviour: `should reject order when stock is zero`
- [ ] Arrange, Act, Assert structure; one behaviour per test
- [ ] Test happy path, edge cases (empty, null, max size) and error cases
- [ ] Mock external services (APIs, queues, email); never call real third parties in unit tests
- [ ] Tests are fast and independent; no test depends on another's order
- [ ] No skipped tests (`.skip` , `xit`) merged without a ticket
- [ ] Integration tests for APIs with a test database (Docker / Testcontainers)

Tools

Stack	Unit tests	Coverage report	API / e2e
Node.js	Jest or Vitest	`--coverage` (lcov)	Supertest, Playwright
React	Vitest + React Testing Library	lcov	Playwright
Python	pytest	pytest-cov (`--cov --cov-report=xml`)	pytest + httpx

Test report every developer attaches to a PR

```
## Test report
- Unit tests: 142 passed, 0 failed
- Coverage on new code: 86%
- Integration tests: passed
- Manual test: steps tried + screenshots
- SonarQube quality gate: Passed
```

CI publishes the coverage report (lcov or coverage.xml) to SonarQube automatically, so reviewers can check it there.

SonarQube and the quality gate

If the SonarQube quality gate fails, the PR cannot be merged. No exceptions without the tech lead's written approval.

Quality gate (on new code)

Metric	Required
New bugs	0
New vulnerabilities	0
Security hotspots reviewed	100%
Coverage on new code	80% or more
Duplicated lines on new code	3% or less
Maintainability rating	A
Reliability rating	A
Security rating	A

Developer rules

- [ ] Install SonarQube for IDE (formerly SonarLint) in VS Code and connect it to the company server, so issues show while you type
- [ ] Fix issues before pushing, not after review
- [ ] Never mark an issue "won't fix" or "false positive" without a lead's review and a comment why
- [ ] Leave the code cleaner than you found it: fix at least one old issue in files you touch

CI setup (GitHub Actions example)

```
- name: Test with coverage
  run: npm run test -- --coverage

- name: SonarQube scan
  uses: SonarSource/sonarqube-scan-action@v5
  env:
    SONAR_TOKEN: ${{ secrets.SONAR_TOKEN }}
    SONAR_HOST_URL: ${{ secrets.SONAR_HOST_URL }}

- name: Quality gate
  uses: SonarSource/sonarqube-quality-gate-action@v1
  timeout-minutes: 5
  env:
    SONAR_TOKEN: ${{ secrets.SONAR_TOKEN }}
```

```
# sonar-project.properties
sonar.projectKey=my-service
sonar.sources=src
sonar.tests=src,tests
sonar.test.inclusions=**/*.test.ts
sonar.javascript.lcov.reportPaths=coverage/lcov.info
sonar.exclusions=**/node_modules/**,**/dist/**
```

Branch protection on main requires the SonarQube check to pass before the merge button works. Check action versions against the SonarSource docs when setting up.

Using Claude Code well

Claude Code gives accurate results when it has project context, a clear task, and a way to check its own work (tests, build, lint). The developer still owns every line it writes. Based on the official Claude Code best practices.

One-time project setup

- [ ] Run `/init` in the repo to generate a starter `CLAUDE.md`, then trim it
- [ ] Commit `CLAUDE.md` to Git so the whole team shares it; keep it short (if a line would not prevent a mistake, delete it)
- [ ] Put in `CLAUDE.md`: build / test / lint commands, our coding rules from this handbook, folder structure, branch and PR conventions, gotchas
- [ ] Block Claude from reading secrets with deny rules in `.claude/settings.json`

- [ ] Add hooks for rules that must always run (e.g. run ESLint after every edit, block edits to migrations)
- [ ] Add shared skills in `.claude/skills/` for repeated tasks (API conventions, fix-a-ticket flow)
- [ ] Add a `security-reviewer` subagent in `.claude/agents/`
- [ ] Install the `gh` CLI so Claude can read issues and open PRs

Example CLAUDE.md

```
# Commands
- npm run dev | npm run test | npm run lint | npm run build

# Rules
- Max 400 lines per file, 40 per function; split if larger
- No hard-coded URLs, keys or magic numbers; use src/config and src/constants
- Layers: routes -> controller -> service -> repository; no SQL outside
repositories
- Every new function needs a unit test; run tests before saying you are done
- IMPORTANT: never read, print or edit .env files or secrets

# Git
- Branch: feature/JIRA-123-name; Conventional Commits; PR under 400 lines
```

Example .claude/settings.json (block secrets)

```
{
  "permissions": {
    "deny": [
      "Read(./.env)",
      "Read(./.env.*)",
      "Read(./secrets/**)",
      "Read(./**/*.pem)",
      "Read(./**/*.key)"
    ]
  }
}
```

Workflow for accurate answers

1. **Explore:** start in plan mode (`Shift+Tab` until plan mode is on). Ask Claude to read the relevant folder first.
2. **Plan:** ask for a plan listing files to change; read it and correct it before any code is written.

3. **Implement:** approve the plan, and ask Claude to write tests, run them and fix failures.
4. **Verify:** ask for evidence (test output, build result, screenshot), not "it's done".
5. **Review:** run `/code-review` or a review subagent in a fresh context, then review the diff yourself.
6. **Commit:** ask Claude to commit with a Conventional Commit message and open a PR.

Skip the plan for one-line fixes; use it when the change touches several files or you do not know the code.

Prompting: bad vs good

Bad	Good
"fix the login bug"	"Users get logged out after token refresh. Check src/auth/, especially refreshToken. Write a failing test first, then fix it and run the tests."
"add tests for orders"	"Write unit tests for @src/modules/orders/orders.service.ts covering zero stock, invalid coupon and payment timeout. Mock the payment client."
"add a new API"	"Add GET /api/v1/invoices following the pattern in @src/modules/orders. Use Zod validation, pagination, and add tests."
"make it better"	"Refactor this file under 400 lines by moving validation and DB queries into separate files; behaviour must not change and tests must pass."

Session habits

- [ ] Use `@file` to point Claude at exact files instead of describing them
- [ ] `/clear` between unrelated tasks; a clean session beats a long, messy one
- [ ] Corrected Claude twice on the same issue? `/clear` and write a better prompt
- [ ] `Esc` to stop and redirect; `Esc` twice or `/rewind` to undo to a checkpoint
- [ ] Use subagents for big investigations so the main session stays focused
- [ ] Name long sessions with `/rename` and resume with `claude --continue`

Safety rules for Claude Code

- [ ] Never paste production data, customer data or real keys into prompts
- [ ] Never point Claude Code at production databases or servers

- [ ] Read every diff before committing; you are the author, not Claude
- [ ] Check every new package it adds actually exists and is maintained
- [ ] Senior review required for AI-written auth, payment, encryption and permission code

Growth path: basic to advanced

Each level builds on the one before. Developers move up when they consistently do everything in their current level without reminders.

Level 1: Basics (every developer, from day one)

- [ ] Git basics: pull, branch, commit, rebase, resolve conflicts, open a PR
- [ ] Follow the 400-line and no-hard-coding rules
- [ ] Never commit secrets; understand .env vs .env.example
- [ ] Write unit tests for own code; attach the test report
- [ ] Pass lint and SonarQube before every push
- [ ] Read error messages and logs before asking for help
- [ ] Use Claude Code with plan mode and review every line

Level 2: Intermediate

- [ ] Design a feature module end to end (routes to repository)
- [ ] Write integration tests and handle edge cases
- [ ] Review others' PRs with useful comments
- [ ] Apply OWASP Top 10 in daily code
- [ ] Use Docker locally; understand the CI/CD pipeline
- [ ] Debug with logs, metrics and traces across services
- [ ] Set up project CLAUDE.md, hooks and skills for the team

Level 3: Advanced (senior)

- [ ] Write design docs and ADRs; run design reviews
- [ ] Plan for scale, failure, cost and security (threat modeling)
- [ ] Own the quality gate and reduce tech debt
- [ ] Lead incident response and blameless postmortems
- [ ] Mentor juniors with weekly code walkthroughs
- [ ] Run the Before Development and Before Launch checklists
- [ ] Set AI coding rules and review AI-heavy PRs

Developer sign-off

Each developer signs once when joining and again when the handbook changes.

Developer	Level	Read and agreed	Date